Sistema de Monitoreo de Redes de Agua Potable empleando Apache Cassandra

Sandy Ruiz Salazar, Jeferson Salazar Zambrana, Estefanía Núñez Jimenéz, keylor Gómez Jiménez, Jimena Díaz González

Escuela de Estadística, Universidad de Costa Rica

***Resumen*—**Este trabajo propone una solución al caso de Aquansence CR mediante un modelo NoSQL de columnas anchas usando Apache Cassandra, ya que esta empresa requiere un sistema para almacenar gran cantidad de lecturas sobre el agua potable proveniente de sus sensores de caudal, presión, temperatura y calidad. El modelo se creó basándose en patrones de consulta. Incluye ocho tablas desnormalizadas. Estas tablas están particionadas por sensor o por área. También están particionadas por día. Las tablas tienen ordenamiento temporal en orden descendente. Se usa compactación por intervalos de tiempo. Esto evita que las particiones sean muy grandes. Se creó un conjunto sintético. Este conjunto tiene 1 000 sensores. Los sensores están distribuidos en 20 áreas. El conjunto contiene 1 000 000 de mediciones. La creación del conjunto fue replicable., Con un 2 % de lecturas anómalas, cargado mediante escritura asíncrona usando sentencias preparadas y lotes no registrados. Las pruebas se hicieron en un clúster de un solo nodo con factor de replicación 1.

Palabras clave—Apache Cassandra, Wide-Column, NoSQL, Telemetría, Series de Tiempo, Bases de Datos, Sensores IoT, Ingestión Masiva de Datos.

I. INTRODUCCIÓN Y PROBLEMA

Las organizaciones que operan infraestructuras críticas, como las redes de distribución de agua potable, enfrentan hoy un crecimiento constante en el volumen de datos generados por sus procesos de monitoreo. Esta tendencia ha impulsado la adopción de bases de datos NoSQL, una alternativa que surge frente a las limitaciones de escalabilidad y flexibilidad de esquema que presentan los modelos relacionales tradicionales cuando deben gestionar grandes volúmenes de información [1].

AquaSense CR es una empresa que monitorea redes de agua potable mediante sensores de caudal, presión, temperatura y calidad instalados en distintas zonas de su infraestructura. Estos dispositivos generan lecturas de forma continua, lo que exige un sistema de almacenamiento capaz de absorber una alta frecuencia de escritura sin degradar su desempeño, además de permitir consultas rápidas de series de tiempo por sensor y por intervalo, incluso cuando el volumen histórico de datos crece de manera sostenida durante meses de operación.

Esta problemática no es exclusiva de AquaSense CR. Experiencias similares en plantas de tratamiento de agua potable describen que los sistemas de monitoreo manual o discontinuo limitan la capacidad de detectar cambios repentinos en los parámetros críticos y dificultan el ajuste oportuno de los procesos operativos [4]. La digitalización de estas infraestructuras mediante sensores y sistemas de almacenamiento especializados representa, por tanto, una alternativa viable para superar dichas limitaciones.

Los modelos relacionales, ajustados para transacciones estructuradas y combinaciones (joins), no están diseñados para escrituras masivas

y constantes como las de una red de sensores. En un caso similar, el almacenamiento de datos de sensores para el monitoreo de estructuras civiles se eligió Cassandra debido a su elevada velocidad de lectura y escritura, y el prototipo mostró un rendimiento notablemente mejor en las consultas de subconjuntos de datos [5].

Este trabajo tiene como objetivo diseñar e implementar una solución de datos basada en el modelo de columnas anchas, utilizando Apache Cassandra, que permita a AquaSense CR ingerir de forma continua las lecturas de sus sensores, consultar eficientemente por sensor, zona y rango de tiempo, y mantener un desempeño estable y escalable a medida que el volumen de datos crece. La selección de esta tecnología se sustenta en evaluaciones previas de rendimiento y escalabilidad de Cassandra [2], así como en estudios sobre el comportamiento de su modelo de consistencia ajustable frente a distintas cargas de trabajo [3], aspectos que se detallan en las secciones siguientes.

II. FUNDAMENTOS DEL MODELO NOSQL (Columnas Anchas)

Las bases de datos NoSQL surgieron como solución a la necesidad de procesar grandes volúmenes de datos crecientes y con estructuras más diversas, en situaciones donde el modelo relacional clásico muestra limitaciones de escalabilidad y rigidez de esquema [1]. Sabemos que las bases de datos relacionales, que estructuran la información en tablas con un esquema rígido y relaciones establecidas, las bases de datos NoSQL se enfocan en la distribución horizontal de los datos y permiten estructuras más versátiles, ajustadas a las necesidades particulares de cada aplicación.

En la clasificación de bases de datos NoSQL, el modelo de columnas anchas (wide-column) se distingue por estructurar la información en filas identificadas por una clave, donde cada fila puede tener un número variable de columnas agrupadas en familias de columnas [2]. A diferencia de una tabla relacional tradicional, no es imprescindible que todas las filas tengan la misma estructura de columnas, lo que proporciona flexibilidad al modelo sin perder la organización tabular.

Cassandra almacena de forma contigua, en memoria y disco, los datos de una misma partición, por lo que las consultas sobre datos continuos dentro de ella (por ejemplo, las lecturas de un sensor en un periodo) resultan especialmente eficientes [5].

A diferencia de los sistemas relacionales, en un modelo de columnas anchas el diseño de las tablas se basa en las consultas que la aplicación debe satisfacer (diseño basado en consultas), en lugar de normalizar los datos según las relaciones entre las entidades. Esto significa que, en numerosas ocasiones, es fundamental replicar datos en varias tablas para mejorar diferentes patrones de acceso, una práctica que en el modelo relacional se vería como un incumplimiento de las formas normales, pero que en este contexto es beneficiosa para aumentar la eficiencia de lectura [5].

Respecto a la consistencia de los datos, las bases de datos de columnas anchas como Apache Cassandra no proporcionan por defecto una consistencia inmediata entre replicas, sino que permiten niveles de consistencia configurables de acuerdo con las necesidades de la aplicación. Esta propiedad, denominada consistencia eventual, permite equilibrar la disponibilidad y el rendimiento del sistema con relación a la precisión total de los datos en todo momento, y su configuración influye de manera directa y cuantificable en la latencia y el throughput del sistema [3].

Finalmente, la escalabilidad horizontal es una característica esencial de este modelo: el sistema puede expandirse incorporando más nodos al clúster, sin que esto requiera interrupciones en el servicio ni una reconfiguración manual complicada, lo que es especialmente importante para aplicaciones que, como el monitoreo de sensores, producen volúmenes de datos en aumento de manera continua a lo largo del tiempo [5].

III. TECNOLOGÍA SELECCIONADA

A. Apache Cassandra 5.0.9

Lo primero, para la implementación de la solución, se optó por Apache Cassandra, una base de datos de código abierto NoSQL que claramente se enmarca en el modelo de columnas anchas, diseñada para manejar grandes volúmenes de datos a través de múltiples nodos. Su arquitectura de tipo peer-to-peer, sin un nodo maestro, elimina los puntos únicos de fallo y permite que cualquier nodo dentro del clúster gestione solicitudes tanto de lectura como de escritura, lo que la hace ideal para un sistema de monitoreo que necesita funcionar de manera continua [2].

Originalmente desarrollada en Facebook y liberada como proyecto open source en 2008, hoy es mantenida por la Apache Software Foundation.

Sus características principales son:

- **Arquitectura peer-to-peer:** todos los nodos son iguales, sin maestro.
- **Escalabilidad horizontal lineal:** agregar nodos aumenta la capacidad proporcionalmente.
- **Alta disponibilidad:** replicación configurable y sin downtime.
- **Consistencia ajustable:** desde ONE hasta ALL.
- **Modelo wide-column:** ideal para series de tiempo.
- **CQL (Cassandra Query Language):** sintaxis similar a SQL.

B. Justificación de la elección

La selección de Cassandra en lugar de otras opciones de columnas anchas está respaldada por varios factores. Primero, las pruebas experimentales de rendimiento indican que la base de datos se escala favorablemente al incrementar el número de nodos, mejorando notablemente los tiempos de respuesta en cargas de lectura y escritura con grandes volúmenes de datos [2]. En segundo lugar, su modelo de consistencia ajustable (configurable a través de niveles como ONE, QUORUM o ALL) permite encontrar un balance entre la disponibilidad del sistema y la precisión de los datos ofrecidos, lo que es una elección de diseño que afecta directamente el rendimiento observado en las evaluaciones [3].

TABLA I: Alternativas consideradas

Alternativa	Motivo del descarte
ScyllaDB	Compatible con CQL, pero con menos documentación y comunidad más pequeña.
Apache HBase	Requiere Hadoop y Zookeeper; mayor complejidad operativa.
Google Cloud Bigtable	Dependencia de la nube; el proyecto debe ser reproducible localmente.
Apache Cassandra	Elegida: open source, madura, ampliamente documentada, fácil de levantar con Docker.

Además, Cassandra es gratuita y de código abierto bajo la licencia Apache 2.0, y se puede desplegar de manera reproducible utilizando contenedores Docker en cualquier sistema operativo que soporte dicha tecnología, cumpliendo así con el requerimiento de que la solución funcione sin depender de licencias costosas ni de credenciales privadas.

C. Versión y entorno

Versión utilizada: se utilizó Apache Cassandra 5.0.9, desplegada mediante Docker en un contenedor único (clúster "AquaSenseCluster"). Para el entorno de desarrollo se empleó la estrategia de replicación SimpleStrategy con factor de replicación 1; para un entorno de producción se documenta el uso recomendado de NetworkTopologyStrategy con factor de replicación 3 (RF=3).

IV. Modelo y Arquitectura

A. Filosofía de modelado

Cassandra es **query-driven**: las tablas se diseñan a partir de las consultas, no de un modelo

normalizado. Cada consulta frecuente tiene su propia tabla desnormalizada, ya que Cassandra no soporta joins eficientes.

Este método sugiere que, antes de crear cualquier tabla, es esencial determinar con exactitud los patrones de acceso que la aplicación deberá manejar. Para AquaSense CR, esos patrones se relacionan con los siete requisitos esenciales establecidos para el caso: verificación de lecturas recientes por sensor, consulta por área y tiempo, identificación de anomalías, y creación de resúmenes consolidados, entre otros.

B. Tablas del modelo

TABLA II: El keyspace aquasense contiene 8 tablas:

Tabla	Propósito	Partition Key	Clustering Key
sensores	Catálogo de sensores	sensor_id	—
sensores_por_zona	Sensores por zona	zona	sensor_id
lecturas_por_sensor	Series de tiempo por sensor	(sensor_id, bucket)	fecha_hora DESC
lecturas_por_zona	Series de tiempo por zona	(zona, bucket)	fecha_hora DESC, sensor_id ASC
anomalias_por_sensor	Anomalías por sensor	(sensor_id, bucket)	fecha_hora DESC
anomalias_por_zona	Anomalías por zona	(zona, bucket)	fecha_hora DESC, sensor_id ASC
resumen_diario_zona	Agregados diarios	zona	dia DESC
ultima_lectura_sensor	Última lectura por sensor	sensor_id	—

Como se puede notar, las tablas lecturas_por_sensor y lecturas_por_zona abordan el mismo tipo de datos (las lecturas de los sensores), aunque divididas según dos criterios de acceso diferentes. Esta intencionada repetición de datos (una acción que se evitaría) en un modelo relacional normalizado es justo lo que permite que ambas consultas (por sensor y por zona) se atiendan con acceso directo a la partición correspondiente, sin requerir operaciones adicionales de filtrado o combinación entre tablas [5].

C. Decisiones clave de diseño

1. **Bucketing diario:** el campo bucket (tipo date) agrupa las lecturas por día para evitar particiones gigantes.

2. **Clustering DESC:** las últimas lecturas se devuelven primero sin ordenamiento adicional.

3. **TimeWindowCompactionStrategy (TWCS):** agrupa SSTables por ventana de 1 día, óptimo para series de tiempo.

4. **Desnormalización:** cada lectura se escribe en lecturas_por_sensor y lecturas_por_zona, aceptando el costo de escritura a cambio de lecturas rápidas.

5. **Tablas dedicadas para anomalías:** evitan el uso de ALLOW FILTERING.

El primer punto (bucketing diario) requiere atención especial, pues aborda directamente uno de los requisitos explícitos del caso: prevenir que una partición acumule un volumen de datos excesivo a medida que se expande el historial de mediciones. Sin este mecanismo, una partición definida exclusivamente por sensor_id o por zona se expandiría de manera indefinida con el tiempo, perjudicando el rendimiento de lectura de esa partición en particular [5].

D. Arquitectura de la solución

La arquitectura consta de cuatro capas:

1. **Capa de sensores:** 1.000 sensores distribuidos en 20 zonas que generan telemetría continua.

2. **Capa de ingesta:** scripts Python que generan y cargan los datos en lotes.

3. **Clúster Cassandra:** nodo único en desarrollo (3 nodos en producción con RF=3).

4. **Capa de consulta:** scripts CQL y Python que sirven las consultas operativas.

E. Escalabilidad y alta disponibilidad

- **Escalabilidad horizontal:** agregar nodos distribuye datos automáticamente mediante consistent hashing.
- **Replicación RF=3:** cada dato existe en 3 nodos.
- **Sin punto único de fallo:** cualquier nodo puede responder consultas.
- **Recuperación:** nodetool repair sincroniza réplicas tras fallos.

Es fundamental señalar que estas características de replicación y resistencia a fallos se refieren a la configuración sugerida para un entorno de producción (RF=3 de varios nodos). El entorno de pruebas empleado en este proyecto funciona con un solo nodo y un factor de replicación de 1, una configuración idónea para el desarrollo reproducible en una única máquina, aunque no permite evidenciar experimentalmente esas capacidades de replicación, esta restricción se aborda en la Sección VIII.

V. IMPLEMENTACIÓN

Se desarrolló un generador (data/generate_data.py) que genera datos sintéticos de forma reproducible utilizando una semilla constante, simulando 1000 sensores distribuidos en 20 zonas que corresponden a cantones o ciudades de Costa Rica, con un volumen mínimo de 1 000 000 de mediciones y un porcentaje fijo del 2% de lecturas anómalas incluidas intencionalmente para validar su detección posterior. El generador también recrea un patrón de consumo diario realista, modificando el caudal según la hora del día a través de una función senoidal que imita una mayor demanda durante el día y una menor en la madrugada.

La carga de datos (data/load_data.py) se llevó a cabo a través de ingesta asíncrona mediante execute_concurrent del controlador oficial de Cassandra para Python, con un nivel de concurrencia ajustable (32 operaciones simultáneas por defecto). Las escrituras se llevan a cabo mediante sentencias preparadas (prepared statements) organizadas en lotes no registrados (UNLOGGED batches) de tamaño ajustable (50 filas por defecto), que se acumulan en un búfer antes de ser enviados al clúster (20 000 filas por defecto). Cassandra está optimizada para cargas intensivas de escritura gracias al uso secuencial del disco [2]. Para aprovecharlo, la carga combina escritura asíncrona concurrente, sentencias preparadas y lotes no registrados agrupados por partición.

Las consultas correspondientes a cada requisito del caso se implementaron en los archivos database/queries/consultas_sensor.cql, consultas_zona.cql, consultas_anomalias.cql y consultas_resumen.cql, ejecutadas a través de la consola cqlsh del clúster.

Las lecturas anómalas se producen a través de una probabilidad constante del 2% que se aplica a cada medición simulada. Cuando una medición se identifica como anómala, se le asigna de manera aleatoria uno de seis tipos posibles de anomalía: fuga, baja presión, alta presión, temperatura elevada, caudal irregular, o mala calidad. Cada clase de anomalía altera los valores de caudal, presión y temperatura de manera realista; por ejemplo, una fuga aumenta el caudal de 1.6 a 2.5 veces su valor original y disminuye la presión entre un 30% y un 60%. Este campo booleano (anomalia) se guarda directamente en las tablas de lecturas y se replica en las tablas específicas de anomalías (anomalias_por_sensor, anomalias_por_zona), lo que elimina la necesidad de la operación ALLOW FILTERING.

VI. PRUEBAS DE DESEMPEÑO Y RESULTADOS

A. Validación funcional.

Se validó el esquema de las 8 tablas con inserciones y consultas de verificación (docs/validacion-schema.md). Además, se ejecutaron 9 pruebas de consultas (tests/test_consultas.py), que incluyen la cantidad de sensores y zonas y el rechazo de consultas sin clave de partición, y 3 pruebas de concurrencia (tests/test_concurrencia.py) en un keyspace aparte. Pasaron las 12. Todas las consultas se resuelven con la clave de partición completa, sin ALLOW FILTERING ni recorridos completos de tabla.

B. Entorno

Windows 11 Home, Intel Core i7-1065G7 (4 núcleos, 8 hilos), 7 959 MB de RAM; Docker con 8 CPUs y 3.711 GiB; contenedor aquasense con Cassandra 5.0.9, heap máximo de 1 024 MB, SimpleStrategy y RF=1. El cliente Python y Cassandra corrieron en la misma máquina. Las latencias se midieron desde Python con sentencias preparadas, 5 repeticiones de calentamiento y 30 medidas por consulta, en tres corridas.

C. Ingesta masiva

Se cargaron 1 000 000 de mediciones de 1 000 sensores en 20 zonas con escritura asíncrona (execute_concurrent, 32 operaciones simultáneas), sentencias preparadas y lotes no registrados de 50 filas agrupados por partición, aprovechando que Cassandra está optimizada para escritura intensiva [2].

TABLA III: ingesta de 1 000 000 de mediciones

Métrica	Valor
Filas cargadas (verificadas)	1 000 000
Tiempo	127.8 s
Velocidad	7 824 filas/s
Lotes enviados	95 866
Anomalías	19 970 (2.00 %)

D. latencia

Tabla IV: Mediana / p95 en ms, desde Python.

Consulta	Filas	Corrida 2	Corrida 3
Última lectura de un sensor	1	2.80 / 3.35	2.37 / 2.66
Últimas 10 lecturas	10	3.08 / 3.92	2.58 / 3.37
Sensor y 1 día	96	4.97 / 5.96	4.55 / 5.59
Sensor y 7 días	664	19.07 / 28.01	13.24 / 16.64
Zona y 1 día	4 579	47.04 / 93.25	38.38 / 67.62
Anomalías de un sensor	2.2 (prom.)	2.87 / 4.36	3.04 / 4.34
Anomalías de una zona	90.6 (prom.)	3.58 / 4.96	3.25 / 3.77
Resumen diario de una zona	7	2.87 / 3.70	2.32 / 2.70

*E. Particionamiento y compactación***.**

Tabla V: Tamaño de las particiones.

Tabla	Particiones	Tamaño máx.	Celdas máx.
lecturas_por_sensor	10 494	6 866 B	642
lecturas_por_zona	220	379 022 B	24 601

Ambas quedan muy por debajo de los 100 MB que el diseño toma como referencia. Con una versión previa del generador de 7 zonas, la partición zona-día llegó a 943 127 bytes y la consulta por zona y día a 201.5 ms de mediana (frente a 38 a 47 ms con 20 zonas), lo que muestra que el tamaño de la partición gobierna esa consulta. Se configuró TimeWindowCompactionStrategy con ventanas

de un día, pero con 11 días de datos no se midió su efecto.

VII. LIMITACIONES

El clúster es un único nodo con SimpleStrategy y RF=1, por lo que no se demuestran la replicación ni la tolerancia a fallos, y los niveles de consistencia ajustable [3] no se diferencian entre sí (el estudio de referencia usa tres nodos y RF=3). En producción se usaría NetworkTopologyStrategy con RF=3. Cliente, servidor y cassandra-stress comparten la misma máquina. La velocidad de carga varió entre 3 477 y 7 938 filas/s, y las filas/s del estrés no son comparables con las del cargador, que escribe en varias tablas. Los datos son sintéticos y cubren 11 días, así que no se observó el crecimiento de las particiones tras meses.

La consulta por zona y día es un punto débil, ya que devuelve 4.579 filas.En un clúster real, todas las escrituras de una zona en un día terminan en la misma partición, lo que puede generar un riesgo de hotspot.Esto se podría mitigar usando un bucket horario o añadiendo un shard adicional.

El generador marca las anomalías, pero el sistema solo las almacena y consulta, sin detectarlas.
Cassandra no es adecuada para realizar agregaciones o análisis ad hoc, y las consultas que recorren muchas particiones afectan el rendimiento [5]. Por eso, el resumen diario se precalcula (en el prototipo, durante la carga).

VIII. CONCLUSIONES

En primera instancia, la solución sobre Apache Cassandra cumplió los requisitos funcionales del caso. El diseño guiado por consultas, con particiones *(sensor_id, bucket*) y (*zona, bucket*), resolvió las consultas por sensor, zona y rango temporal con la clave de partición completa y sin ALLOW FILTERING. Las consultas de hasta 96 filas respondieron con mediana de 2.3 a 5.0 ms, y la de zona y día (4 579 filas) con 38 a 47 ms. El bucketing diario mantuvo las particiones pequeñas (máximo de 379 022 bytes por zona-día y 6 866 bytes por sensor-día), y la carga de 1 000 000 de mediciones tomó 127.8 s (7 824 filas/s). Pasaron las 12 pruebas funcionales y la prueba de estrés no produjo errores.

Además, la consulta por área fue la debilidad, y el costo del diseño es la desnormalización: cada lectura se registra en diferentes tablas y no se evaluó su impacto en la escritura, tampoco se realizó comparación entre TimeWindowCompactionStrategy y otras estrategias.

A modo de cierre, los hallazgos están restringidos a un nodo con RF=1, información sintética y anomalías señaladas por el generador (Sección VII). Se recomienda para trabajos futuros contemplar un clúster de tres nodos con RF=3, un histórico de meses, un bucket horario para la tabla por región, la identificación de anomalías en la ingesta y una API de consultas.

IX. REFERENCIAS

[1] H. A. Herrera y C. Rueda Valenzuela, «NoSQL, la nueva tendencia en el manejo de datos», Tecnol. Investig. Academia TIA, vol. 4, n.º 1, pp. 147–150, may 2016.

[2] M. Barata and J. Bernardino, "Cassandra's performance and scalability evaluation," in Proc. 5th Int. Conf. Data Management Technologies and Applications (DATA), 2016, pp. 127-134, doi: 10.5220/0005980101270134.

[3] A. Gorbenko, A. Romanovsky, and O. Tarasyuk, "Interplaying Cassandra NoSQL consistency and performance: A benchmarking approach," in Dependable Computing – EDCC 2020 Workshops (Communications in Computer and Information Science, vol. 1279), 2020, pp. 168-184, doi: 10.1007/978-3-030-58462-7_14.

[4] Á. H. Santamaría Masapuncho y M. M. Bayas Altamirano, «Monitoreo y evaluación de parámetros de calidad de agua obtenidos por la Internet de las cosas (IoT) para la planta de tratamiento de agua potable el carrizal, perteneciente a la parroquia San Miguel, del Cantón Salcedo, provincia de Cotopaxi», *Rev. InGlobal*, vol. 4, n.º 2, pp. 280-300, nov. 2025, doi: 10.62943/rig.v4n2.2025.380.

[5] J. Llanos Fariña, «Implementación de un sistema de almacenamiento de datos masivos para monitoreo estructural», Memoria de título, Dept. Ing. Informática, Univ. de Concepción, Concepción, Chile, 2018.

ENLACE AL REPOSITORIO DE CÓDIGO

Repositorio GitHub:

https://github.com/sandykrs/AquaSense-Cassandra-Grupo5.git